On the Subject of Non-Equivocation

Defining Non-Equivocation in Synchronous Agreement Systems

Mads Frederik Madsen
IT University of Copenhagen
Copenhagen, Denmark
mfrm@itu.dk

Søren Debois
IT University of Copenhagen
Copenhagen, Denmark
debois@itu.dk

ABSTRACT

We study *non-equivocation* in synchronous agreement protocols: the restriction on faulty processes that they cannot act differently towards distinct non-faulty processes. Guarantees of non-equivocation have been used to provide improved fault tolerance in agreement protocols, and various mechanisms for achieving it have been proposed. However, the exact meaning of non-equivocation varies subtly in the literature. In this paper, we propose two different formal notions of non-equivocation: *strong* and *weak*. We define both as fault models for synchronous agreement protocols with reliable channels, and we show how the two models yield distinct bounds for the minimal number of communication rounds required and the maximum number of faulty processes tolerable to achieve agreement: 1 round, $n > t$ for strong non-equivocation; and $t + 1$ rounds, $n > 2t$ for weak non-equivocation. This makes weak non-equivocation the only fault model with a lower bound on fault tolerance of $n > 2t$ for broadcast agreement and interactive consistency, confirming the folklore knowledge that equivocation is, in a sense, the most critical of the Byzantine faults. Finally, we show how the weak and strong non-equivocation fault models relate to well-known agreement problems: strong non-equivocation corresponds to Byzantine broadcast and weak non-equivocation to crusader agreement.

CCS CONCEPTS

• **Theory of computation → Distributed algorithms**.

KEYWORDS

Agreement, non-equivocation, fault tolerance, synchronous systems

ACM Reference Format:
Mads Frederik Madsen and Søren Debois. 2020. On the Subject of Non-Equivocation: Defining Non-Equivocation in Synchronous Agreement Systems. In *ACM Symposium on Principles of Distributed Computing (PODC '20), August 3–7, 2020, Virtual Event, Italy.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3382734.3405731

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODC '20, August 3–7, 2020, Virtual Event, Italy
© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-7582-5/20/08…$15.00
https://doi.org/10.1145/3382734.3405731

1 INTRODUCTION

Equivocation is the act of a faulty process *acting differently* when correct behaviour is to be consistent. In this paper, we study *non-equivocation* in synchronous agreement protocols: the guarantee that otherwise Byzantine processes will never equivocate.

The notion of equivocation was introduced informally and without the name by Lamport et al. [25]. Subsequently, Chun et al. [11] named it and showed that assuming non-equivocation (in one form) allows for an improvement in fault tolerance. Mechanisms providing non-equivocation can be constructed in different ways, with the most well-known being perhaps Attested Append-Only Memory (A2M) [11] and Trusted Incrementers (TrInc) [26].

Studies of non-equivocation have used subtly different definitions of the concept, leading to subtly different results. E.g.,[27] proposes the non-equivocation based "f-resilient" condition for agreement in synchronous systems, and conjecture informally that this property will be enough also in asynchronous ones. Meanwhile, [12] proves that transferable authentication is a necessary addition to non-equivocation for a fault tolerance improvement in asynchronous systems. The result of [12] does not contradict the conjecture in [27] as the two papers are referring to different notions of non-equivocation.

Thus, we propose two different concepts of non-equivocation: *strong* and *weak*. Both require faulty processes to not "lie differently", but weak non-equivocation allows faulty processes to selectively omit messages to some participants, where strong non-equivocation does not. In the above example [27] refers to a notion like that of strong non-equivocation, while [12] refers to one like that of weak non-equivocation.

Both A2M and TrInc can provide at best weak non-equivocation since a malicious or faulty process using a trusted module can still choose to simply not send messages. On the other hand, partial broadcast channels [13, 21, 22, 34], and local broadcast channels [23, 24] provide strong non-equivocation, as they guarantee broadcast messages are reliably and identically received by all neighbours.

Technically, we work in the original framework of Pease et al. [31], where we define both weak and strong non-equivocation as fault models for synchronous systems with reliable channels. We show for both how to solve the agreement problem of *interactive consistency* [31]. For strong non-equivocation, this is trivial; we obtain a 1-round, $n > t$ fault-tolerant algorithm simply by broadcasting a value, where n is the number of processes, and t is the number of faulty processes. For weak non-equivocation, we show how the problem is solvable using the algorithm of Pease et al. [31] with only minor adjustments. In this case, $t+1$ rounds are necessary, and we obtain exactly a fault-tolerance of $n > 2t$.

We relate the present results to the existing body of work on agreement protocols in Table 1 on page 3. To the best of our knowledge, weak non-equivocation is the only fault model with a lower bound of $n > 2t$ on the fault tolerance of broadcast agreement and interactive consistency in a synchronous model with reliable channels. This result demonstrates formally how equivocation is at the core of impossibility of Byzantine fault tolerance.

The notions of weak and strong non-equivocation as fault models fit neatly with our existing understanding of agreement problems in the following sense. Under guarantees of strong non-equivocation, it is trivial to implement Byzantine broadcast; conversely, given a Byzantine broadcast oracle, it is trivial to obtain strong non-equivocation. Similarly, under guarantees of weak non-equivocation, it is trivial to solve crusader agreement [15]; and conversely, weak non-equivocation is straightforward to obtain given a crusader agreement oracle. Essentially, properties of non-equivocation in synchronous models come down to properties of an oracle for the corresponding agreement problem.

The weak non-equivocation fault model resembles the authenticated Byzantine fault model in that both can be achieved by the use of cryptographic mechanisms. However, we show that while they do share a subspace of runs not in the omission fault model, both also contain a subspace of runs that the other fault model does not. So as fault models they overlap, but only partly.

Overview. We proceed as follows. After presenting related work, we give in Section 2 a system model, in particular recalling the framework of Pease et al. [31]. We introduce strong non-equivocation in Section 3, followed by weak non-equivocation in Section 4. In Section 5 we give bounds on rounds and fault tolerance, and in Section 6 we prove those bounds tight. Finally, we discuss relations to other agreement problem variants in Section 7. For want of space, some proofs and technical details have been omitted; they can be found in an on-line appendix [29].

Related work. In *agreement protocols* processes attempt to agree on one or more values. Processes are given an input value, a *private value*, and must agree on one or more of these values, the *decision value(s)*. Table 1 shows optimal fault tolerance for agreement problems under various fault models, including results of the present paper. The notion of (authenticated) Byzantine agreement problems was introduced by Pease, Shostak, and Lamport [31][25], formulated as interactive consistency and the Byzantine generals problem. In interactive consistency, the non-faulty processes must agree on a value for each process, while in the Byzantine generals problem (i.e. Byzantine broadcast agreement) the non-faulty processes must agree on the private value of a pre-specified process. Interactive consistency can be achieved by running an instance of broadcast agreement per process [19]. Consensus is the problem formulation where every process suggests a value, and non-faulty processes decide on a single common value. It notably differs from interactive consistency and broadcast agreement in the validity property [3, 19]: if all non-faulty processes suggest the same value, then all non-faulty processes must decide that value. This stronger validity property makes the problem impossible to solve, unless $n > 2t$, when processes are allowed to lie about their value. Consensus can be achieved, if $n > 2t$, by running a majority function on an interactive consistency vector; similarly, broadcast agreement can be achieved by running a consensus algorithm on the received values of a broadcast [19]. Uniform consensus differs from the other agreement problems in that also *faulty* processes must agree on the decided value [33, 35]. Fault-tolerant agreement is generally impossible under asynchrony (FLP-impossibility), as shown by Fischer, Lynch, and Paterson [20] because non-faulty processes cannot distinguish a faulty process from delayed messages. This impossibility is commonly circumvented with either partial synchrony [18], relaxation of termination or determinism [5, 7, 32] or the use of failure-detectors [9, 10].

Non-equivocation was suggested by Chun et al. [11] as a means of improving fault-tolerance, exemplified via improvements to PBFT by Castro and Liskov [8]. The non-equivocation mechanism used was an attested append-only log (A2M). Levin et al. [26] showed how to construct such a log more efficiently using trusted increment-only counters (TrInc). Non-equivocation is strongly linked to broadcast primitives, and such two primitives have been used to achieve non-equivocation; partial broadcast channels [13, 21, 22, 34] and local broadcast channels [23, 24]. Here, non-equivocation means that messages sent through special channels are received reliably and identically by all neighbours. This corresponds to our notion of *strong non-equivocation* in Section 3 but limited to subsets of all processes. Other non-equivocation notions, e.g. that of [1, 4, 11, 12, 14, 27], are weaker system-wide properties, where faulty processes may omit messages. This corresponds to our notion of *weak non-equivocation* in Section 4. Algorithms that use non-equivocation to achieve agreement with improved fault-tolerance include [1, 14, 27].

Fitzi and Maurer [21] showed that synchronous Byzantine broadcasts (i.e. Byzantine generals) can be achieved with $n > 2t$ using a partial broadcast mechanism, which provides a property akin to strong non-equivocation, between only subsets of 3 processes. Chun et al. [11] showed how non-equivocation can be used to implement an $n > 2t$ resilient PBFT solution. Clement et al. [12] showed that neither non-equivocation nor transferable authentication is enough to give an $n > 2t$ resilient agreement protocol under asynchrony with relaxed termination; both are needed. Lastly, Jaffe et al. [22] examined the trade-off between equivocation and redundancy by showing under which conditions a fault-tolerance of $n = 2t + h$, for any $h > 0$, can be achieved using partial broadcast channels providing (strong) non-equivocation.

2 SYSTEM MODEL AND NOTATION

A system comprises n synchronous processes connected with synchronous and reliable point-to-point channels. Up to t processes may be faulty, and can deviate arbitrarily from the protocol but not break the guarantees awarded by the fault model. To simplify the presentation, we separate the message-exchange protocol from the rest of the algorithm. By slight abuse of language, we will refer to the message-exchange as *the protocol*, and the calculation made by a process on its received messages as *the algorithm*.

We write $P = \{p, q, r, \ldots\}$ for the set of all processes, N for the subset of all non-faulty processes, and F for the subset of faulty processes: $N \cup F = P$, $N \cap F = \emptyset$, $|N| = n$, and $|F| = t$. As usual, P^* denotes all finite strings over P, P^+ denotes all non-empty finite strings, and we refer to an element of P^* as a *process-sequence*.

Table 1: Optimal fault tolerance of different agreement problems in different fault models, assuming synchronous processes and reliable, synchronous channels. Rows titled in bold represent results of the present paper.

Fault model	Broadcast agreement	Interactive consistency	Consensus	Uniform consensus
Byzantine	$n > 3t$ [25]†	$n > 3t$ [31]	$n > 3t$ [25]‡	N/A
Weak non-equivocation	$n > 2t$	$n > 2t$	$n > 2t$	N/A
Authenticated Byzantine	$n > t$ [25]	$n > t$ [31]	$n > 2t$ [36]	N/A
Omission	$n > t$ [35]	$n > t$ [35]	$n > t$ [35]	$n > 2t$ [33]
Crash	$n > t$ [28]	$n > t$ [28]	$n > t$ [28]	$n > t$ [35]
Strong non-equivocation	$n > t$	$n > t$	$n > 2t$	N/A

† Aka Byzantine generals

‡ Aka Byzantine agreement

n processes, t faulty

Each message sent comprises a value $v \in V$, and an element of P^+, representing the route the value has been sent along. Each process can correctly determine the immediate sender of a message. Note that lying about the process-sequence of a message is equivalent to lying about the value [31], so we consider only lies about values for the remainder of this paper.

We will define non-equivocation as a fault model for the protocol. A fault model limits which messages faulty processes may send. For this purpose, we use the framework of [31], where the message-exchange is represented as a map $\sigma : P^+ \to V$ from process-sequences to values. For instance, $\sigma(p|q) = v$ means that q has sent v to p, ("|" denotes string concatenation); and $\sigma(p|q|r) = v$ that q has reported to p that r has reported the initial value v to q. Processes do not send messages to themselves, so, abusing notation, we consider only process-sequences which do not contain consecutive duplicate characters, e.g., we disregard $p|p|q$.

A process p obviously has access only to the messages it has received. Formally, we define σ_p by $\forall w \in P^* \colon \sigma_p(w) = \sigma(p|w)$. For ease of notation, and since process-sequences cannot include consecutive duplicates, we let $\sigma_p(p|w) = \sigma_p(w)$. In cases of message omissions, we use $\perp$ to denote the absence of a value in σ. We use $\langle\perp\rangle$ as the value specifying the absence of a value, that is, if $\sigma(p|r) = \perp$ then a correct p will report $\langle\perp\rangle$ to any q: $\sigma(q|p|r) = \langle\perp\rangle$.

2.1 Interactive consistency

We recall the *interactive consistency problem* [31]. Each process p starts with a private value v_p and must compute a vector, vec_p, containing a value for each process such that:

- **Agreement:** All non-faulty processes compute the same vector: $\forall p, q \in N \colon vec_p = vec_q$
- **Validity:** If q is non-faulty, all non-faulty processes sets the q'th element in the vector to q's private value: $\forall p, q \in N \colon vec_p[q] = v_q$
- **Termination:** All non-faulty processes must eventually compute a vector.

Interactive consistency is known to be reducible to Byzantine broadcast and vice versa [19], so to solve interactive consistency it is sufficient to give an algorithm describing Byzantine broadcast. We exploit this equivalence to interchangeably solve one or the other, depending on which is more convenient. We generally find it more convenient to solve Byzantine broadcast, however, interactive consistency supports a more intuitive induction proof in Section 5.

For Sections 3 to 5 , we use the following specialized round-based message-passing protocol: all processes go through $t+1$ *information exchange broadcast rounds*. In the first broadcast round, the non-faulty processes broadcast their private values, and subsequently, in round r they broadcast the values they received in round $r-1$, along with the route that value has taken so far. As we consider only a finite number of rounds and synchronous channels and processes, *termination* of the protocol is trivial, and we need only consider the termination of the algorithm.

This protocol gives us the following guarantee: a non-faulty process neither lies about its private value, nor its previously received messages.

Definition 2.1 (Non-faulty process guarantee). *A non-faulty process truthfully broadcast values, reporting $\langle\perp\rangle$ only when they have observed a message omission:*

$$\forall q \in N : \forall p \in P \setminus \{q\} : \forall w \in P^* : \quad \sigma(p|q|w) = \begin{cases} \langle\perp\rangle, & \text{if } \sigma(q|w) = \perp \\ \sigma(q|w), & \text{otherwise} \end{cases}$$

In the non-faulty process guarantee, $\langle\perp\rangle$ can be interpreted as the reporting process reporting that there exists a suffix of the process-sequence of that message, w', for which $\sigma(w') = \perp$. Note that $\langle\perp\rangle$ is simply a value, and *can* be reported as any process' private value. Also, note that this interpretation is not a guarantee; a faulty process *can* report $\langle\perp\rangle$ having received neither $\perp$ nor $\langle\perp\rangle$ in the previous round. *Faulty* processes are allowed to deviate arbitrarily from the protocol and so from Definition 2.1; they are restricted only by the fault model.

3 STRONG NON-EQUIVOCATION

We define the strong non-equivocation fault model as a weakening of the Byzantine fault model:

Definition 3.1 (Strong non-equivocation). *For any given process-sequence, all processes—faulty or not—report the same value to all other processes:*

$$\forall p, r, r' \in P \colon \forall w \in P^* \colon \sigma(r|p|w) = \sigma(r'|p|w)$$

Since w can be the empty string and is prefixed with the same p on both sides of the equality, this is equivalent[1] *to*

$$\forall r, r' \in P\colon \forall w \in P^+\colon \sigma(r|w) = \sigma(r'|w)$$

Under strong non-equivocation, a faulty process may still lie: a faulty process q may receive v from The strong non-equivocation fault model overlaps with the crash fault model: a faulty process may fail by not sending any messages from a given round onwards, thereby acting as if crashed. However, the strong non-equivocation fault model does not *contain* the crash fault model: in the crash fault model a faulty process can fail during its broadcast, leading to a situation where some processes have received a message from the crashing process, while others have not; a violation of Definition 3.1. Conversely, a faulty process may lie consistently in the strong non-equivocation fault model, but not in neither the general omission- nor the crash fault model. So the strong non-equivocation fault model must overlap with a strict subspace of the crash fault model, but also extend into a strict subspace of the Byzantine fault model.

Given a guarantee of strong non-equivocation, it is trivial to solve *Byzantine broadcast*, and so also interactive consistency. For source q, each correct p simply decides on the value it was sent by q in round 1: $vec_p[q] = \sigma_p(p|q)$. Clearly, if q is correct, each correct process sets the same value. If q is faulty, then by Definition 3.1, each other process has received the same message from q, and so all decide on the same value. (We give a formal proof of correctness in the online appendix [29].) Note that the strong non-equivocation fault model really is very strong: all is done in one round, and within that single round, the source process *cannot fail* in a way that makes it in any way distinguishable from a correct process, making interactive consistency and Byzantine broadcast solvable for $n > t$. Note that for the problem of consensus, the lower bound is $n > 2t$, as processes can lie about their private value (see Section 1).

We argue that Byzantine broadcast and strong non-equivocation are interchangeable in the sense that strong non-equivocation implies a trivial solution to Byzantine broadcast as shown above. Similarly, a Byzantine broadcast oracle can be used to make algorithms assuming the strong non-equivocation fault model work in the Byzantine fault model. Informally, the latter holds because, after Byzantine broadcast, all non-faulty processes agree on some value for the source process, which is exactly strong non-equivocation.

Strong non-equivocation is most useful in systems with partial trust, or given primitives providing the properties to parts of the system. For examples, work on partial broadcast channels, e.g. [13, 21, 22, 34], use a definition of non-equivocation much like strong non-equivocation: every process on the partial broadcast channel receive the same messages. Using this primitive, Fitzi and Maurer [21] consider the problem of achieving global broadcast, i.e. Byzantine broadcast, from such strongly non-equivocating partial broadcast channels. Given that strong non-equivocation is interchangeable with Byzantine broadcast, the problem of achieving global broadcast from partial broadcast channels can be rephrased as a problem of *extending* strong non-equivocation across systems; achieving system-wide strong non-equivocation from strongly non-equivocating partial broadcast channels.

[1] We are implicitly understanding this requirement to be only when $r'|w$ contains no consecutive duplicates, see Section 2.

4 WEAK NON-EQUIVOCATION

Key practical approaches to non-equivocation revolve around using cryptographic primitives in trusted modules, possibly hardware, making equivocation readily detectable by non-faulty processes. This is the approach taken by TrInc and A2M [11, 26]. In this case, strong non-equivocation is not an appropriate fault model, since the possibly faulty process employing the trusted module may still omit any message it chooses, including omitting messages to some but not all other processes within a single round. Thus, we define *weak non-equivocation*, where all processes still must report the same value to other processes for each process-sequence, but are allowed to omit messages to some subset of processes. Recall absence of a message is represented as $\perp$ in σ.

Definition 4.1 (Weak non-equivocation). *For any given process-sequence, all processes—faulty or not—report the same value to a subset of processes, and omit messages to the rest:*

$$\forall p, r, r' \in P\colon \forall w \in P^*\colon \sigma(r|p|w) \neq \perp \wedge \sigma(r'|p|w) \neq \perp \implies \sigma(r|p|w) = \sigma(r'|p|w)$$

Since w can be the empty string and is prefixed with the same p on both sides of the equality, this is equivalent to:

$$\forall r, r' \in P\colon \forall w \in P^+\colon \sigma(r|w) \neq \perp \wedge \sigma(r'|w) \neq \perp \implies \sigma(r|w) = \sigma(r'|w)$$

Observe that the omission- and crash fault models are strict subspaces of the weak non-equivocation fault model: a faulty process can arbitrarily omit any message, and consequently also stop sending messages altogether from a given round onwards, thereby acting as if crashed. See Fig. 4 on page 9 for an illustration of the fault model spaces.

4.1 Algorithm for $t = 1$

To establish intuition about the general algorithm, we give in Algorithm 1 on page 5 a solution to Byzantine broadcast under weak non-equivocation when $t = 1$. (For the general case, see Section 5.) Note that for $n = 3$ this is the traditional Byzantine generals problem, which is *not* solvable in the Byzantine fault model [25].

The algorithm proceeds as follows over two rounds. If a non-$\perp$ value is received in the first round (line 3), we decide on that value (line 4). Otherwise, the source must be faulty and (as $t = 1$) all other processes non-faulty. If any other non-faulty process received a value from the source, they correctly report that in the second round (line 5), and we decide on that value (line 6). Otherwise, no non-faulty process can have received a value in the first round, and so we decide on $\langle\perp\rangle$ as the value. The non-faulty processes all set the same value because they consider only the value sent by the source in the first round, and by weak non-equivocation (Definition 2.1), all who received the a value in round 1 received the same value. Altogether, we have correctness (for full proof, refer to the generalisation in Theorem 5.1):

Theorem 4.2. *In the weak non-equivocation fault model, interactive consistency is solvable for any $n > 1$ if $t \leq 1$.*

It is instructive to consider the different possibilities a faulty process has for lying under weak non-equivocation in the case

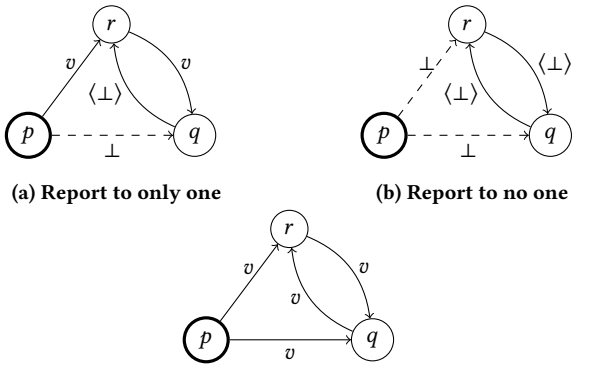


Figure 1: Three examples of $n = 3, t = 1$ executions, showing all possible actions a faulty source, p, can take under weak non-equivocation, parametric in the value of v.

$n = 3, t = 1$. Refer to Figs. 1 and 2. In each case, it is immediately apparent to non-faulty processes either who is faulty, or what the correct value should be. E.g., in Fig. 1a, r knows to decide on v upon receiving it; q knows that p is faulty from the omission, thus deciding upon the value from r, which must be correct because $t = 1$. In Fig. 2b, r knows that q is faulty as p could not have reported u to q without violating weak non-equivocation (Definition 4.1).

4.2 Properties of weak non-equivocation

To reason about the general case, we introduce the formal device of *consistent majorities*. Consider a process p trying to decide on the value of a source process q. A consistent q-majority for a value v is a set Q of more than t processes, including q, who have consistently reported the value v to p, both for themselves and for each other.

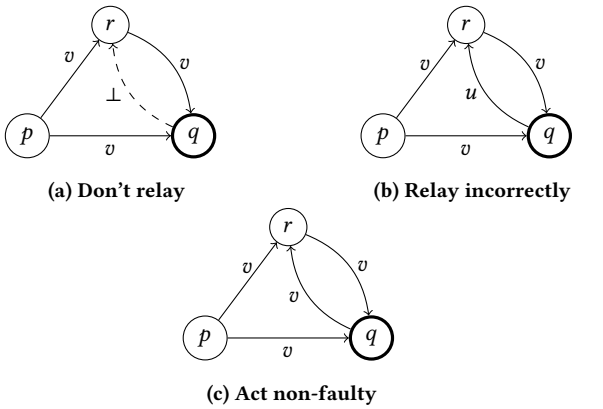


Figure 2: Three examples of $n = 3, t = 1$ executions, showing all possible actions a faulty non-source, q, can take under weak non-equivocation, parametric in the values of v and u and assuming that $v \neq u$.

Algorithm 1 Algorithm for process p to set q'th value in vector

function $\mathrm{WNE}_{t=1}(\sigma_p, p, q)$

1: **if** $p = q$ **then**
2: return $\sigma_p(p)$ ▷ p is q
3: **else if** $\sigma_p(p|q) \neq \perp$ **then**
4: return $\sigma_p(p|q)$ ▷ q reported a value to p
5: **else if** $\exists r.\sigma_p(p|r|q) \neq \langle\perp\rangle$ **then**
6: return $\sigma_p(p|r|q)$ ▷ Someone else told p about q
7: **else**
8: return $\langle\perp\rangle$ ▷ q have reported $\perp$ to everyone
9: **end if**

end function

Definition 4.3 (consistent majority). *Write* $\mathrm{maj}^{q,v}_{p,Q}(t, \sigma_p, P)$ *and say that p has a consistent q-majority for v, when*

$$\forall w \in Q^* : |w| \leq t : \sigma_p(p|w|q) = v \wedge |Q| \geq t + 1 \wedge Q \subseteq P$$

Note that p having a consistent q-majority does not imply that any other process has a consistent q-majority.

A consistent majority is related to the notion of a *quorum* but differs by (1) the source process must be part of a consistent majority, and (2) the processes in a consistent majority Q reports that everybody in Q agree on the same value, to p. The principle behind consistent majorities was described in [31].

Just from the message exchange σ, we can derive helpful relationships between the distribution of faulty processes and the existence of consistent majorities, *provided* we may assume $n > 2t$. Recall that $P = N \cup F$ is the set of n processes, where F is the set of at most t faulty processes, and N is the set of non-faulty processes. Let p, q and r be processes and let σ be a map of messages after at least $t + 1$ information exchange rounds according to Definition 2.1 and in the weak non-equivocation fault model (Definition 4.1).

Lemma 4.4. *If p is non-faulty, then all non-faulty processes will have a consistent majority for p's private value v_p:*

$$\forall p, q \in N : \exists v \in V : \exists Q \subseteq P : \mathrm{maj}^{p,v_p}_{q,Q}(t, \sigma_q, P)$$

Proof. By Definition 2.1, all non-faulty processes correctly forward values each round. So p must have reported v_p to the set of non-faulty processes N in the first round, and all processes in N must have reported v_p to each other in all following rounds. Because $|N| > t$, the set N must be a consistent majority for v_p. □

Lemma 4.5. *If a process p does not have a consistent q-majority then q must be faulty:*

$$\forall p, q \in P : (\forall v \in V : \forall Q \subseteq P : \neg\mathrm{maj}^{q,v}_{p,Q}(t, \sigma_p, P)) \implies q \in F$$

Proof. Contraposition of Lemma 4.4. □

Lemma 4.6. *If p has a consistent q-majority for v, then it cannot also have a consistent q-majority for a distinct value u:*

$$\forall p, q \in P : \forall v, u \in V :$$
$$(\exists Q, R \subseteq P : \mathrm{maj}^{q,v}_{p,Q}(t, \sigma_p, P) \wedge \mathrm{maj}^{q,u}_{p,R}(t, \sigma_p, P)) \implies v = u$$

Proof. By contradiction. Assume that for some process p, $\mathsf{maj}^{q,v}_{p,Q}(t,\sigma_p,P)$, $\mathsf{maj}^{q,u}_{p,R}(t,\sigma_p,P)$ and $v \neq u$ holds. By definition of consistent majority (Definition 4.3), any set Q s.t. $\mathsf{maj}^{q,v}_{p,Q}(t,\sigma_p,P)$ must include q, so it must be the case that $\sigma(p|q) = v$. But then by $\mathsf{maj}^{q,u}_{p,R}(t,\sigma_p,P)$ it must be the case that $\sigma(p|q) = u$, which contradicts $v \neq u$. □

Lemma 4.7. *No consistent majority for $\bot$ can exist:*

$$\forall p,q \in P\colon \forall v \in V\colon (\exists Q \subseteq P\colon \mathsf{maj}^{q,v}_{p,Q}(t,\sigma_p,P)) \implies v \neq \bot$$

Proof. By contradiction. Assume that there exists a Q s.t. $\mathsf{maj}^{q,\bot}_{p,Q}(t,\sigma_p,P)$. By Definition 4.3, $|Q| > |F|$, so there must exist a non-faulty process in Q, i.e. a non-faulty process that has reported $\bot$. But, by Definition 2.1, non-faulty processes never report $\bot$. □

Lemma 4.8. *If non-faulty processes p and q each have a consistent r-majority, then those must be for the same value:*

$$\forall p,q \in N\colon \forall r \in P\colon \forall v,u \in V\colon$$
$$(\exists Q,R \subseteq P\colon \mathsf{maj}^{r,v}_{p,Q}(t,\sigma_p,P) \wedge \mathsf{maj}^{r,u}_{q,R}(t,\sigma_q,P)) \implies v = u$$

Proof. *By contradiction.* Assume that p and q have consistent r-majorities for respectively different values v and u. By Definition 4.3, p having an consistent r-majority for v implies that $\sigma(p|r) = v$, and q having an consistent r-majority for u implies $\sigma(q|r) = u$. By weak non-equivocation (Definition 4.1), $\sigma(p|r) \neq \sigma(q|r)$ implies that either v or u is $\bot$. But by Lemma 4.7, no consistent majority for $\bot$ can exist: a contradiction. □

Algorithm 2 Algorithm for process p to set source process q's value in an interactive consistency vector

```
    function PSL(σ_p, n, t, p, q, P)
1:      if p = q then                                  ▷ p's own private value
2:          if σ_p(p) = ⊥ then                         ▷ ⊥ ∉ V, so choose default ⟨⊥⟩
3:              return ⟨⊥⟩
4:          else
5:              return σ_p(p)
6:          end if
7:      end if
8:      if ∃Q ⊆ P: maj^{q,v}_{p,Q}(t, σ_p, P) then     ▷ Consist. maj. for v
9:          return v
10:     end if                          ▷ No consist. maj.; q is faulty (Lemma 4.5)
11:     σ̂_p ← {}                                       ▷ Start constructing σ̂_p
12:     P̂ ← P \ {q}                                    ▷ Exclude q from σ̂_p
13:     for all w ∈ P̂*, s.t. p|w|q is in the domain of σ_p do
14:         σ̂_p(p|w) ← σ_p(p|w|q)
15:     end for                                        ▷ Done constructing σ̂_p
16:     v̂ec ← [ ]                                      ▷ Constructing v̂ec: IC vector of σ̂_p
17:     for all r ∈ P̂ do
18:         v̂ec[r] ← PSL(σ̂_p, n − 1, t − 1, p, r, P̂)
19:     end for                                        ▷ Done constructing v̂ec
20:     if ∃v ≠ ⟨⊥⟩ : |{r | v̂ec[r] = v}| ≥ t then
21:         return v                    ▷ Min. one non-faulty process received v
22:     else
23:         return ⟨⊥⟩
24:     end if
    end function
```

5 SOLVING INTERACTIVE CONSISTENCY UNDER WEAK NON-EQUIVOCATION

To solve interactive consistency, it is sufficient to define an algorithm that achieves agreement, validity and termination for one source process, and then run that algorithm for each process in the system [19]. Assuming $n > 2t$ and weak non-equivocation, we give such an algorithm in Algorithm 2, which is a generalisation of Algorithm 1 and an adaptation of the algorithm by Pease, Shostak, and Lamport [31], which assumes the Byzantine fault model. The key differences are the addition of $\bot$ and $\langle\bot\rangle$, the required size of consistent majorities in Definition 4.3, and the size of the set in Line 20, which is adapted to fit with a fault tolerance of $n > 2t$.

Algorithm 2 relies on two insights. (1) If a process cannot find a consistent majority, then the source process must be faulty (Lemma 4.5). (2) After $t+1$ rounds, if the source process is faulty, we can treat the other processes reports about the source process as their private values in a new instance of interactive consistency; write $\widehat{\sigma}$ for the restriction of σ where we remove any mention of the source. This $\widehat{\sigma}$ will have t message rounds regarding these values. Since the *faulty* source process is excluded, $\widehat{\sigma}$ can be used to recursively find the values reported to the rest of the processes, by either finding a consistent majority for them or recursively applying the same method. In the base case, there is only one faulty process left, in which case Algorithm 2 behaves like Algorithm 1.

We proceed to prove correctness of Algorithm 2.

Theorem 5.1. *Algorithm 2 is correct.*

Proof sketch (full proof in the online appendix [29]). **Validity**. Assume a non-faulty source p; we must show that for any non-faulty process, Algorithm 2 returns the private value v_p. Because p is non-faulty, all non-faulty processes have a consistent p-majority for v_p (Lemma 4.4), and cannot have a consistent p-majority for another value (Lemma 4.6). Thus Algorithm 2 will return v_p in Line 9. **Agreement**. We must show that for all non-faulty processes, Algorithm 2 returns the same value. Consider a source p. If p is non-faulty, agreement follows trivially from validity (line 9), so assume p faulty. We proceed by induction on t. For the base case, $t = 1$, p either reports a value to some processes, in which case all others know to trust a forwarded value (line 21), or not report at value at all, in which case everybody agrees that this is the case (line 23), and choose a default value. For the induction case, consider two arbitrary non-faulty processes, and proceed by cases on which have a consistent p-majority. If they both do, then it is for the same value by Lemma 4.8. If neither has a consistent p-majority, then both know that p is faulty, and run Algorithm 2 recursively with p, obtaining a correct interactive consistency vector by IH. They both run a majority function on this vector (line 20), arriving at the same result. If one has a consistent majority, and the other does not, then the latter must have a recursive vector with a majority of the same value as that of the consistent majority by Definition 4.3 and IH (see also Lemma C.1 in the online appendix [29]). **Termination**. Immediate from the observation that message exchange is bounded at $t+1$;

the domain of σ is then finite; the domain of σ_p is then also finite; and it is now obvious that all loops in Algorithm 2 terminates. □

Theorem 5.1 shows that interactive consistency can be solved for $n > 2t$ under weak non-equivocation. However, the solution is not efficient, requiring $\Theta(n^{t+1})$ messages to construct the necessary σ. Algorithms that solve interactive consistency in a polynomial number of messages, e.g. [16] exists for the Byzantine fault model. We leave a polynomial algorithm for the weak non-equivocation fault model as future work.

6 TIGHTNESS

Having proved interactive consistency possible for $n > 2t$ under weak non-equivocation, we now show that this lower bound is tight.

Theorem 6.1. *Given any finite number of rounds, k, and $n \leq 2t$, no synchronous deterministic algorithm exists which, given a σ in the weak non-equivocation fault model, always finds an interactive consistency vector with the properties of agreement, validity and termination.*

Proof. By contradiction. Assume a deterministic algorithm, A, where any process p can run $A(p, \sigma_p, q)$ and obtain a value for q fulfilling the requirements of agreement and validity. Without loss of generality assume that $n = 2t$ and that each run continues for k rounds, where $k \geq t + 1$. Partition all processes into sets O, Q, R and S, s.t. $|O| = \lfloor \frac{t}{2} \rfloor$, $|Q| = \lceil \frac{t}{2} \rceil$, $|R| = \lceil \frac{t}{2} \rceil$ and $|S| = \lfloor \frac{t}{2} \rfloor$. Note that all processes in any two partitions can be simultaneously faulty, except for Q and R as their combined size may be greater than t.

Let the source process be $p \in O$. We construct four runs with different faulty partitions, such that some non-faulty process will be unable to distinguish two specific runs from each other, leading to a contradiction if the algorithm achieves both validity and agreement. Let v and u be *distinct* values not equal to $\perp$ or $\langle \perp \rangle$. For ease of notation let $A(Q, \sigma_{Run1}, p)$ denote the value each process in Q will decide for p in run 1. We construct runs such that $A(Q, \sigma_{Run1}, p) = v$ by validity, and $A(Q, \sigma_{Run1}, p) = A(Q, \sigma_{Run3}, p)$ by determinism of A. Similarly we will construct the runs such that $A(R, \sigma_{Run2}, p) = u$ by validity, and $A(R, \sigma_{Run2}, p) = A(R, \sigma_{Run4}, p)$ by determinism of A. Lastly we will construct the runs such that $A(S, \sigma_{Run3}, p) = A(S, \sigma_{Run4}, p)$ by determinism of A, ultimately showing that S must decide both v and u which is impossible as $v \neq u$.

We construct the runs as follows (see Fig. 3 for a visual representation).

Run 1 (The processes of R and S are faulty):
- p broadcasts the value v in the first round.
- In the second round, the processes in R broadcast the value u and the processes in S broadcast $\langle \perp \rangle$.
- In the third round the processes in R broadcast the value u and the processes in S broadcast $\langle \perp \rangle$, as the respective values received from O.
- All processes correctly report all other messages.

By validity, all processes in Q decide v.

Run 2 (The processes of Q and S are faulty):
- p broadcasts the value u in the first round.
- In the second round the processes in Q broadcast the value v and the processes in S broadcast $\langle \perp \rangle$.
- In the third round the processes in Q broadcast the value v and the processes in S broadcast $\langle \perp \rangle$, as the respective values received from O.
- All processes correctly report all other messages.

By validity, all processes in R decide u.

Run 3 (The processes of O and R are faulty):
- In the first round p broadcasts the value v to all except the processes in partition S, resulting in S recording $\perp$.
- In the second round the processes in R broadcast u, and O broadcasts v to all but S, resulting in S recording $\perp$.
- In the third round the processes in R broadcast the value u as the values received from O.
- All processes correctly report all other messages.

For the processes in Q, this run is indistinguishable from run 1: in both runs, they have received v from O, u from R, and $\langle \perp \rangle$ from S by the end of round 2, and all partitions will act the same for the remaining rounds. By determinism, Q should decide v to be the value for p. By agreement, S must then decide v to be the value for p.

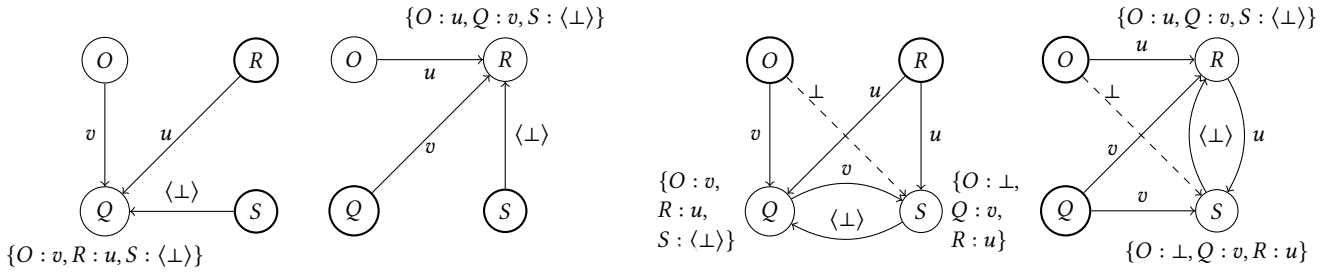
Run 4 (The processes of O and Q are faulty):
- In the first round p broadcasts the value u to all except for the processes in partition S, resulting in S recording $\perp$.
- In the second round processes in Q broadcast v, and O broadcasts u to all but S, resulting in S recording $\perp$.
- In the third round the processes in Q broadcast the value v as the values received from O.
- All processes correctly report all other messages.

For the processes in R, this run is indistinguishable from run 2: in both runs, they have received u from O, v from Q, and $\langle \perp \rangle$ from S by the end of round 2, and all partitions will act the same for the remaining rounds. By determinism, R should decide u to be the value for p. By agreement, S must then decide u to be the value for p. But for the processes in S, this run is indistinguishable from run 3: in both runs, they have received nothing from any process in O, v from all processes in Q, and u from all processes in R by the end of round 2, and all partitions will act the same in the two runs for the remaining rounds. So, by determinism all processes in S should decide v to be the value for p, leading to the contradiction that the processes in S should decide both v and u for distinct v and u. □

Note that this proof uses determinism only to restrict that non-faulty processes decides the same value under indistinguishable runs. This restriction may be implied by the validity and agreement properties, making the proof have no requirement of determinism. We leave a thorough investigation of this idea as future work.

The weak non-equivocation fault model is, to the best of our knowledge, the only fault model with this lower bound fault-tolerance for interactive consistency and broadcast in the synchronous model; the Byzantine fault model has $n > 3t$ and all others has $n > t$ (see Table 1). This uniqueness confirms formally the folklore knowledge that equivocation is the most critical of the Byzantine fault and what necessitates the triple replication of processes in Byzantine agreement problems.

Considering the fault tolerance of general omission algorithms ($n > t$), Theorem 6.1 also suggests that the type of faults that



(a) Run 1 **(b) Run 2** **(c) Run 3** **(d) Run 4**

Figure 3: Runs described in the proof of Theorem 6.1. Faulty partitions are bold. Arrows are selected messages sent in round 2. The source process p is in O, so the arrows from O are messages in both round 1 and 2. For selected partitions, the messages received by end of round 2 are annotated. Note how partition Q receives the same messages in run 1 and 3, partition R receives the same messages in run 2 and 4, and partition S receives the same messages in run 3 and 4.

separates general omission from weak non-equivocation, i.e. lying without equivocating, is what necessitates anything but the least amount of replication. This hypothesis is also backed up by the fault tolerance of the authenticated Byzantine fault model ($n > t$), where faulty processes are prevented from lying *at all* about (but not omitting) the messages of non-faulty processes.

A proof sketch of the tightness of the bound of $t + 1$ rounds is simply showing that weak non-equivocation encompasses the crash fault model. As it is known that the broadcast agreement cannot be solved for less than $t + 1$ rounds in the crash fault model [2], it follows that the problem requires at least as many rounds in weak non-equivocation. A formal proof can be found in the online appendix [29].

7 OTHER MODELS

In this section, we relate weak non-equivocation to (1) the problem of crusader agreement as defined by Dolev [15], and (2) the authenticated Byzantine fault model as defined by Pease et al. [31].

7.1 Crusader Agreement

From weak non-equivocation to crusader agreement. Weak non-equivocation (Definition 4.1) implies a trivial solution to the problem of *crusader agreement* [15]. Crusader agreement is an agreement problem weaker than Byzantine broadcast: non-faulty receiving processes must decide a value in agreement with all other non-faulty processes *unless* the receiving process *knows* that the sending process is faulty. Formally, processes in the crusader agreement problem decides a value for the process p's private value v_p, with these properties:

Crusader agreement: All non-faulty processes decide the same value unless they know that p is faulty.[2]

Validity: If process p is non-faulty, then all non-faulty processes decide v_p.

Termination: All non-faulty processes must eventually decide on a value, or know that p is faulty.

These properties are *exactly true* for any message in the weak non-equivocation fault model: for some message, a non-faulty process either receives a value from the sending process or it receives nothing; if the receiving process receives nothing, then it knows that the sending process is faulty; if the process receives a value, then it is guaranteed that every other non-faulty process that also received a value, have received the same value. Thus, a procedure where the non-faulty processes (1) decides on the received value if they receive one, and (2) otherwise sets the source process a faulty, solves crusader agreement under weak non-equivocation.

From Crusader Agreement to Weak non-equivocation. Suppose there exists an oracle producing crusader agreement upon request, and that we use this oracle to produce crusader agreement for the value of some source process p. After finishing deciding such a value, all processes will (knowingly) be either faulty; non-faulty and having decided a common value; or non-faulty and knowing that p is faulty. This is exactly the same three states a process will find itself in after p has broadcast a value under weak non-equivocation: for some v, all correct processes q will have either $\sigma_q(p) = v$ or $\sigma_q(p) = \bot$; in the latter case, q knows for a fact that p is faulty.

Dolev [15] showed that crusader agreement has a fault tolerance of $n > 3t$ in the pure Byzantine fault model, but that it only requires a constant of 2 information exchange rounds. Dolev's crusader agreement algorithm works by using a purifying function much like Definition 4.3[3] to agree on a value or know that the source is faulty. Assuming that $n > 3t$, we can use Dolev's crusader agreement algorithm as a primitive to map any σ with $t + 2$ rounds in the Byzantine fault model into a σ with $t + 1$ rounds in the weak non-equivocation fault model. The mapping proceeds by using the extra (last) round as the second round in Dolev's algorithm, thus achieving crusader agreement for the values transmitted in round $t + 1$. These values can then be used to achieve crusader agreement for the values transmitted in round t, and so on.

Altogether, it seems any algorithm that operates correctly given a crusader agreement primitive, can replace that primitive with

[2] "know that p is faulty" is a slightly imprecise definition. For more precision, the phrase "can prove that if the receiving process itself is non-faulty, then p must be faulty" can be used.

[3] The most notable difference between Definition 4.3 and Dolevs purifying function is that the purifying function does not require the source process to be part of the consistent majority, and, of course, that the consistent majority must be of size at least $2t + 1$, due to the lower fault tolerance.

an assumption of weak non-equivocation. Examples of algorithms using such a primitive can be found in [6, 17, 30].

7.2 The Authenticated Byzantine Fault Model

In this section, we examine the weak non-equivocation fault model's relationship with the authenticated Byzantine fault model. We refer to the authenticated Byzantine fault model as defined in Pease et al. [31], which says that a faulty process cannot lie (but can *omit* messages) about the values of non-faulty processes:

$$\forall p \in N\colon \forall w', w \in P^*\colon \sigma(w'|p|w) = \sigma(p|w) \vee \sigma(w'|p|w) = \perp\ .$$

Given that interactive consistency problem is solvable in the authenticated Byzantine fault model for $n > t$, and that the same problem is solvable for weak non-equivocation for a tight $n > 2t$, one would expect the authenticated fault model is the strictly stronger fault model. However, we shall see that whereas the authenticated fault model and weak non-equivocation obviously have an intersection, they also have a non-empty set difference in either direction. Clement et al. [12] showed in the asynchronous setting that both authentication and non-equivocation is needed to achieve a fault tolerance of $n > 2t$ for agreement problems, so this relationship between the fault models is not entirely surprising. See Fig. 4 for an Euler diagram of spaces of σ's that exist in different fault models.

We demonstrate that (1) there exists σs where algorithms for weak non-equivocation can solve agreement, but algorithms for the authenticated Byzantine fault model cannot; (2) there exists σs where algorithms for the authenticated Byzantine fault model can solve agreement, but algorithms for weak non-equivocation cannot; and (3) there exists σs where algorithms for either fault model can solve agreement, but algorithms for general omission cannot. Examples of each follow here:

(1) In such a σ, no process can equivocate, but some process must lie about the value of a non-faulty process. Let $n = 3, t = 1$, $p, r \in N$ and $q \in F$. Let r be the source process, and broadcast v in the first round. Then let q proceed to report that they received u in the next round. Any algorithm for authenticated Byzantine faults will wrongly conclude that r is faulty, as only faulty processes can have their value spoofed, while any algorithm for weak non-equivocation will correctly conclude q is faulty, as r has not equivocated.

(2) In such a σ, no process can lie about the value of a non-faulty process but some process must equivocate. Let $n = 3, t = 1$, $p, q \in N$ and $r \in F$. Let r be the source process, and send a v to p, and a u to q in the first round, and let p and q proceed to report their received values. Any algorithm for authenticated Byzantine faults will correctly conclude that r is faulty, as it assumes that different values can only exists for faulty processes, while any algorithm for weak non-equivocation will wrongly conclude the other non-faulty process is, in fact, faulty, as it assumes that r is unable to equivocate.

(3) In such a σ, the following three properties must hold: (I) Some process must lie, so that general omission algorithms cannot solve agreement. (II) No process can equivocate, so that weak non-equivocation algorithms can solve agreement. (III) No faulty process can lie about the values of non-faulty processes so that authenticated Byzantine algorithms can solve agreement. From (I), (II) and (III), we can deduce that $t \geq 2$, and, by the lower bound fault tolerance of weak non-equivocation, that $n \geq 5$: Let $n = 5, t = 2$, $p, q, r \in N$ and $s, x \in F$. Let s be the source process, and send v to p in the first round and nothing, $\perp$, to q and r. Let x act as if it received u, but otherwise, let every process report correctly for the proceeding rounds[4]. Both weak non-equivocation and authenticated Byzantine algorithms will deduce that s is faulty, and that there is no majority for any specific value, and so decide some default value. Meanwhile, for a general omission algorithm, q and r know that s is faulty, but have received the contradicting values v and u, and will thus behave in an unspecified manner.

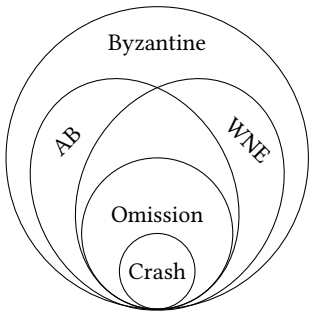


Figure 4: The relationship between spaces of σs described by various fault models. Abbreviations: AB – authenticated Byzantine, WNE – weak non-equivocation. Not pictured is strong non-equivocation, which overlaps with but is not contained in the crash fault model, yet contained in the weak non-equivocation fault model.

8 FUTURE WORK

Our results on fault-tolerance together with that of [12] suggests that the property of *transferable authentication* alone can bridge the gap between an agreement algorithm with non-equivocation in a synchronous system and an asynchronous system with weakened termination. An examination of this result, and in which other system models this translation is valid, could lead to more efficient translations between synchronous and asynchronous algorithms.

Given that the strong non-equivocation fault model is overlapping with the crash fault model, and that it implies a trivial solution to Byzantine agreement as seen in Section 3, it seems that FLP-impossibility may not apply in systems using a primitive providing strong non-equivocation. This opens up questions of whether any weakening of termination, determinism or asynchrony is strictly needed in asynchronous agreement systems that use strong non-equivocation primitives such as partial broadcast channels.

[4]For the sake of argument, let r and q receive u from x before they receive v from p, to prevent early stopping general omission algorithms from deciding v before receiving the contradicting u value

9 CONCLUSION

In this paper, we addressed and formalised the notion of *non-equivocation* in synchronous agreement protocols. We have proposed two different notions of non-equivocation: *strong* and *weak*. Both require faulty processes to not "lie differently", however, weak non-equivocation allows faulty processes to selectively omit messages to some participants, whereas strong non-equivocation does not. We defined both formally as fault models for synchronous agreement protocols with reliable channels, and we showed how the two models yield distinct round- and fault tolerance-bounds for agreement: 1 round, $n > t$ for strong non-equivocation; and $t + 1$ rounds, $n > 2t$ for weak non-equivocation. This makes weak non-equivocation the only fault model with a lower bound on fault tolerance of $n > 2t$ for broadcast agreement and interactive consistency, thus confirming formally the folklore knowledge that equivocation is somehow the most critical of the Byzantine faults. Finally, we have shown how the weak and strong non-equivocation fault models relate to well-known agreement problems: strong non-equivocation corresponds to Byzantine broadcast and weak non-equivocation to crusader agreement.

REFERENCES

[1] Ittai Abraham, Srinivas Devadas, Danny Dolev, Kartik Nayak, and Ling Ren. 2017. Efficient Synchronous Byzantine Consensus. *arXiv:1704.02397 [cs]* (Sept. 2017). arXiv:cs/1704.02397 http://arxiv.org/abs/1704.02397

[2] Marcos Kawazoe Aguilera and Sam Toueg. 1999. A Simple Bivalency Proof That T-Resilient Consensus Requires T+1 Rounds. *Inform. Process. Lett.* 71, 3 (1999), 155–158.

[3] Chagit Attiya, Danny Dolev, and Joseph Gil. 1984. Asynchronous Byzantine Consensus. In *Proceedings of the Third Annual ACM Symposium on Principles of Distributed Computing (PODC '84)*. Association for Computing Machinery, New York, NY, USA.

[4] Michael Backes, Fabian Bendun, Ashish Choudhury, and Aniket Kate. 2014. Asynchronous MPC with a Strict Honest Majority Using Non-Equivocation. In *Proceedings of the 2014 ACM Symposium on Principles of Distributed Computing (PODC '14)*. Association for Computing Machinery, Paris, France, 10–19.

[5] Michael Ben-Or. 1983. Another Advantage of Free Choice (Extended Abstract): Completely Asynchronous Agreement Protocols. In *Proceedings of the Second Annual ACM Symposium on Principles of Distributed Computing (PODC '83)*. Association for Computing Machinery, Montreal, Quebec, Canada, 27–30.

[6] Malte Borcherding. 1997. Byzantine Agreement with Limited Authentication. In *Safe Comp 96*, Erwin Schoitsch (Ed.). Springer, London, 404–413.

[7] Gabriel Bracha and Sam Toueg. 1985. Asynchronous Consensus and Broadcast Protocols. *J. ACM* 32, 4 (Oct. 1985), 824–840.

[8] Miguel Castro and Barbara Liskov. 1999. Practical Byzantine Fault Tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI '99)*. USENIX Association, New Orleans, Louisiana, USA, 173–186.

[9] Tushar Deepak Chandra, Vassos Hadzilacos, and Sam Toueg. 1996. The Weakest Failure Detector for Solving Consensus. *J. ACM* 43, 4 (July 1996), 685–722.

[10] Tushar Deepak Chandra and Sam Toueg. 1996. Unreliable Failure Detectors for Reliable Distributed Systems. *J. ACM* 43, 2 (March 1996), 225–267.

[11] Byung-Gon Chun, Petros Maniatis, Scott Shenker, and John Kubiatowicz. 2007. Attested Append-Only Memory: Making Adversaries Stick to Their Word. In *Proceedings of Twenty-First ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)*. Association for Computing Machinery, Stevenson, Washington, USA, 189–204.

[12] Allen Clement, Flavio Junqueira, Aniket Kate, and Rodrigo Rodrigues. 2012. On the (Limited) Power of Non-Equivocation. In *Proceedings of the 2012 ACM Symposium on Principles of Distributed Computing (PODC '12)*. Association for Computing Machinery, Madeira, Portugal, 301–308.

[13] Jeffrey Considine, Matthias Fitzi, Matthew Franklin, Leonid A. Levin, Ueli Maurer, and David Metcalf. 2005. Byzantine Agreement Given Partial Broadcast. *J Cryptology* 18, 3 (July 2005), 191–217.

[14] Miguel Correia, Giuliana S. Veronese, and Lau Cheuk Lung. 2010. Asynchronous Byzantine Consensus with 2f+1 Processes. In *Proceedings of the 2010 ACM Symposium on Applied Computing (SAC '10)*. Association for Computing Machinery, Sierre, Switzerland, 475–480.

[15] Danny Dolev. 1981. *The Byzantine Generals Strike Again*. Technical Report. Stanford University, Stanford, CA, USA.

[16] Danny Dolev, Michael J. Fischer, Rob Fowler, Nancy A. Lynch, and H. Raymond Strong. 1982. An Efficient Algorithm for Byzantine Agreement without Authentication. *Information and Control* 52, 3 (March 1982), 257–274.

[17] Danny Dolev and Avi Wigderson. 1983. On the Security of Multi-Party Protocols in Distributed Systems. In *Advances in Cryptology*. Springer US, Boston, MA.

[18] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the Presence of Partial Synchrony. *J. ACM* 35, 2 (April 1988), 288–323.

[19] Michael J. Fischer. 1983. The Consensus Problem in Unreliable Distributed Systems (a Brief Survey). In *Foundations of Computation Theory (Lecture Notes in Computer Science)*, Marek Karpinski (Ed.). Springer Berlin Heidelberg, 127–140.

[20] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. 1985. Impossibility of Distributed Consensus with One Faulty Process. *J. ACM* 32, 2 (April 1985), 374–382.

[21] Mattias Fitzi and Ueli Maurer. 2000. From Partial Consistency to Global Broadcast. In *Proceedings of the Thirty-Second Annual ACM Symposium on Theory of Computing (STOC '00)*. Association for Computing Machinery, Portland, Oregon, USA, 494–503.

[22] Alexander Jaffe, Thomas Moscibroda, and Siddhartha Sen. 2012. On the Price of Equivocation in Byzantine Agreement. In *Proceedings of the 2012 ACM Symposium on Principles of Distributed Computing (PODC '12)*. Association for Computing Machinery, Madeira, Portugal, 309–318.

[23] Muhammad Samir Khan, Syed Shalan Naqvi, and Nitin H. Vaidya. 2019. Exact Byzantine Consensus on Undirected Graphs under Local Broadcast Model. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC '19)*. Association for Computing Machinery, Toronto ON, Canada, 327–336.

[24] Muhammad Samir Khan and Nitin H. Vaidya. 2019. Byzantine Consensus under Local Broadcast Model: Tight Sufficient Condition. *arXiv:1901.03804 [cs]* (Jan. 2019). arXiv:cs/1901.03804 http://arxiv.org/abs/1901.03804

[25] Leslie Lamport, Robert Shostak, and Marshall Pease. 1982. The Byzantine Generals Problem. *ACM Trans. Program. Lang. Syst.* 4, 3 (July 1982), 382–401.

[26] Dave Levin, John R. Douceur, Jacob R. Lorch, and Thomas Moscibroda. 2009. TrInc: Small Trusted Hardware for Large Distributed Systems. In *Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation (NSDI'09)*. USENIX Association, Boston, Massachusetts, 1–14.

[27] Chuanyou Li, Michel Hurfin, Yun Wang, and Lei Yu. 2016. Towards a Restrained Use of Non-Equivocation for Achieving Iterative Approximate Byzantine Consensus. In *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS'16)*. IEEE, 710–719.

[28] Nancy A. Lynch. 1996. *Distributed Algorithms*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

[29] Mads Frederik Madsen and Søren Debois. 2020. On the Subject of Non-Equivocation: Defining Non-Equivocation in Synchronous Agreement Systems. (2020). http://itu.dk/people/debois/papers/podc20.pdf

[30] Stephen R. Mahaney and Fred B. Schneider. 1985. Inexact Agreement: Accuracy, Precision, and Graceful Degradation. In *Proceedings of the Fourth Annual ACM Symposium on Principles of Distributed Computing (PODC '85)*. Association for Computing Machinery, Minaki, Ontario, Canada, 237–249.

[31] M. Pease, R. Shostak, and L. Lamport. 1980. Reaching Agreement in the Presence of Faults. *J. ACM* 27, 2 (April 1980), 228–234.

[32] Michael O. Rabin. 1983. Randomized Byzantine Generals. In *24th Annual Symposium on Foundations of Computer Science (SFCS'83)*. 403–409.

[33] Philippe Raipin Parvedy and Michel Raynal. 2003. Uniform Agreement Despite Process Omission Failures. In *2003 IEEE International Parallel and Distributed Processing Symposium (IPDPS'03)*. IEEE.

[34] D. V. S. Ravikant, V. Muthuramakrishnan, V. Srikanth, K. Srinathan, and C. Pandu Rangan. 2004. On Byzantine Agreement over (2,3)-Uniform Hypergraphs. In *18th International Symposium on Distributed Computing (DISC'18)*, Rachid Guerraoui (Ed.). Springer, Berlin, Heidelberg, 450–464.

[35] Michel Raynal. 2002. Consensus in Synchronous Systems: A Concise Guided Tour. In *2002 Pacific Rim International Symposium on Dependable Computing, 2002 (PRDC'02)*. 221–228.

[36] Michel Raynal. 2010. *Fault-Tolerant Agreement in Synchronous Message-Passing Systems*. Morgan & Claypool.